

DSA FROM SCRATCH TO ADVANCED

DAY 11

Binary Search — Implementation & Problems

Concept → Example → Implementation → Complexity → Pattern →
Problems → Solutions → Takeaway

Goal: Learn to implement Binary Search confidently and recognize
where the pattern can be applied.

C++ • Interview Preparation • LeetCode

01 How to use this guide

Binary Search is not just a single algorithm. It is a reusable problem-solving pattern. The core idea is to repeatedly eliminate half of a search space instead of checking every element one by one.

Learning path

Start with the basic sorted-array version. Then understand the loop invariant, edge cases, complexity, and finally extend the idea to common interview patterns.

By the end, you should be able to:

- Explain why Binary Search requires an ordered search space.
- Write iterative Binary Search in C++ without memorizing blindly.
- Trace every step using low, mid, and high.
- Handle duplicates, missing values, and boundary cases.
- Recognize when a problem is actually “Binary Search on the answer”.
- Apply the pattern to real-world scenarios and LeetCode problems.

Core progression

Concept → Why → How → C++ → Dry Run → Complexity → Pattern → Problems → Solutions → Interview Tip → Tomorrow

02 What is Binary Search?

Binary Search is a searching algorithm that finds a target in a sorted search space by repeatedly checking the middle element and discarding the half that cannot contain the answer.

Suppose we have:

```
[2, 5, 8, 12, 16, 23, 38, 56, 72]
```

If the target is 23, we do not need to inspect every value. We inspect the middle, decide whether the target is left or right, and continue only in that half.

The key idea

Divide the search space

At each step, roughly half of the remaining candidates are eliminated. This is why the running time is logarithmic: $O(\log n)$.

Important: the classic form assumes the data is sorted in a way that lets us decide which half can still contain the target. For an ascending array, if $\text{arr}[\text{mid}] < \text{target}$, move right; if $\text{arr}[\text{mid}] > \text{target}$, move left.

03 Why do we need it?

A linear search checks elements sequentially. That is fine for small data, but the number of checks grows directly with the input size.

Comparison

Approach	Sorted data?	Worst-case time	Idea
Linear Search	Not required	$O(n)$	Check one by one
Binary Search	Required	$O(\log n)$	Discard half each step

Real-world intuition

- Dictionary lookup: when words are ordered, you can jump toward the middle rather than start from page 1.
- Phone contacts: a sorted name list lets you jump close to the expected region.
- Database/index concepts: ordered indexes allow efficient narrowing of a search range.
- Finding a threshold: many optimization problems ask for the smallest/largest value satisfying a condition—another form of binary search.

04 How does it work?

For an ascending array, maintain three values: low, mid, and high. Initially, low = 0 and high = n - 1.

At every iteration:

- Compute mid safely: $\text{low} + (\text{high} - \text{low}) / 2$.
- If $\text{arr}[\text{mid}] == \text{target}$ → answer found.
- If $\text{arr}[\text{mid}] < \text{target}$ → target must be to the right, so $\text{low} = \text{mid} + 1$.
- If $\text{arr}[\text{mid}] > \text{target}$ → target must be to the left, so $\text{high} = \text{mid} - 1$.
- If $\text{low} > \text{high}$ → the target does not exist.

Visual dry run: target = 23

```
Index: 0 1 2 3 4 5 6 7 8
Value: 2 5 8 12 16 23 38 56 72
           ^ target
```

- 1) low=0, high=8 → mid=4 → arr[4]=16. Since $16 < 23$, discard indices 0-4.
- 2) low=5, high=8 → mid=6 → arr[6]=38. Since $38 > 23$, discard indices 6-8.
- 3) low=5, high=5 → mid=5 → arr[5]=23. Found!

05 C++ implementation

A clean iterative implementation is usually preferred in interviews because it is simple and uses $O(1)$ auxiliary space.

```
#include <iostream>
#include <vector>
using namespace std;

int binarySearch(const vector<int>& arr, int target) {
    int low = 0;
    int high = arr.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target)
            return mid;

        if (arr[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }

    return -1;
}
```

Example usage

```
vector<int> arr = {2, 5, 8, 12, 16, 23, 38};
int index = binarySearch(arr, 23);

cout << index; // 5
```

Interview habit

Do not rush to code. First state the precondition: "The array/search space is sorted or has a monotonic property." Then explain low, mid, high.

06 Dry run & edge cases

Example 1 — target exists

```
arr = [3, 7, 11, 18, 25, 31, 42]
target = 31

low=0, high=6, mid=3 → 18 < 31 → low=4
low=4, high=6, mid=5 → 31 == 31 → return 5
```

Example 2 — target does not exist

```
arr = [3, 7, 11, 18, 25, 31, 42]
target = 20

mid=3 → 18 < 20 → low=4
mid=5 → 31 > 20 → high=4
mid=4 → 25 > 20 → high=3
low > high → return -1
```

Edge cases to test

- Empty array.
- One element: target present and target absent.
- Target at the first index.
- Target at the last index.
- Target smaller than the minimum.
- Target larger than the maximum.
- Duplicate values, when the problem asks for first/last occurrence.

07 Time & Space Complexity

If the search space has n elements, after each comparison it becomes approximately $n/2$, $n/4$, $n/8$, ... until one candidate remains.

We want k such that $n / 2^k \leq 1$. Therefore $2^k \geq n$, so $k = O(\log n)$.

Time complexity	Best case: $O(1)$ — the first middle element is the target. Average/worst case: $O(\log n)$.
Space complexity	Iterative implementation: $O(1)$ extra space. Recursive implementation: $O(\log n)$ call stack in the worst case.

Why $\log_2(n)$ feels powerful

For 1,000,000 sorted elements, binary search needs only around 20 halving steps in the worst case (because 2^{20} is over one million), while linear search can require up to one million comparisons.

08 The Binary Search pattern

The most important interview skill is not memorizing the basic code. It is recognizing the pattern.

Pattern A — Exact search

Question: "Is target x present?" Use equality and move left/right based on comparison.

Pattern B — Boundary search

Question: "What is the first/last position satisfying a condition?" Keep searching after finding a valid answer.

Pattern C — Binary Search on the answer

Question: "What is the minimum/maximum value for which a condition becomes true?" The values themselves may not be in an array. What matters is that the feasibility condition is monotonic.

Recognition test

Ask: "If I know the answer is too small, can I conclude all smaller answers are also invalid?" If yes, there may be a monotonic search space suitable for binary search.

09 LeetCode Problem 1 — Binary Search

Problem: Given a sorted array of distinct integers and a target, return its index if found; otherwise return -1.

Approach

This is the classic exact-search pattern. Compare the target with `arr[mid]` and eliminate one half each iteration.

```
int search(vector<int>& nums, int target) {
    int low = 0, high = nums.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (nums[mid] == target) return mid;
        if (nums[mid] < target) low = mid + 1;
        else high = mid - 1;
    }

    return -1;
}
```

Example: `nums = [-1,0,3,5,9,12]`, `target = 9` → `answer = 4`.

Takeaway

If the input is sorted and the question asks whether a value exists, first think of classic Binary Search.

10 LeetCode Problem 2 — Search Insert Position

Problem: Given a sorted array of distinct integers and a target, return the index if found. If not found, return the position where it should be inserted.

Key insight

Even when the target is absent, the final low pointer lands at the first position where target could be inserted.

```
int searchInsert(vector<int>& nums, int target) {
    int low = 0, high = nums.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (nums[mid] == target) return mid;
        if (nums[mid] < target) low = mid + 1;
        else high = mid - 1;
    }

    return low;
}
```

Example: [1,3,5,6], target=2 → return 1.

Example: [1,3,5,6], target=7 → return 4.

Pattern upgrade

The result is not always “found/not found”. Sometimes the final boundary itself is the answer.

11 LeetCode Problem 3 — First & Last Position

Problem: Given a sorted array with duplicates, return the first and last position of a target.

Why one search is not enough

When you find target, do not immediately return. To find the first occurrence, record the index and continue searching left. For the last occurrence, continue searching right.

```
int firstPosition(vector<int>& nums, int target) {
    int low=0, high=(int)nums.size()-1, ans=-1;

    while(low<=high) {
        int mid = low + (high-low)/2;
        if(nums[mid] == target) {
            ans = mid;
            high = mid - 1; // keep going left
        } else if(nums[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return ans;
}
```

Run the same idea again for the last position, but when `nums[mid] == target` set `ans = mid` and move `low = mid + 1`.

Complexity

Two binary searches → $O(\log n) + O(\log n) = O(\log n)$ time, $O(1)$ extra space.

12 Real-world problem: finding a price threshold

Imagine an e-commerce platform has sorted shipping prices by delivery speed. A user asks: “What is the first shipping option costing at least ₹500?”

Data

```
Prices: [199, 299, 399, 499, 599, 799, 999]
Target: 500
```

A linear scan checks 199 → 299 → 399 → 499 → 599. Binary search can jump to the middle, decide whether the answer lies left or right, and converge on 599.

The same concept appears in many systems:

- First available appointment after a requested time.
- First version/build that introduced a bug.
- Minimum server capacity that can handle a workload.
- Smallest budget that satisfies a set of requirements.
- First timestamp at which a metric crosses a threshold.

Important

The data does not always have to be an array. If the search space is ordered and the condition is monotonic, Binary Search may still apply.

13 Binary Search on the Answer

This is one of the most valuable extensions for interviews. Suppose the answer is an integer from 1 to 1,000,000. Instead of testing every number, ask whether a candidate value is feasible.

Example: minimum eating speed

Given piles of bananas and H hours, find the minimum integer speed k such that all bananas can be eaten within H hours.

Search space: $k \in [1, \max(\text{pile})]$. Feasibility: "Can we finish within H hours at speed k?" If speed k works, every speed greater than k also works. That is monotonic.

```
bool canFinish(vector<int>& piles, int h, int k) {
    long long hours = 0;
    for (int bananas : piles) {
        hours += (bananas + k - 1) / k;
        if (hours > h) return false;
    }
    return true;
}

// Binary search the minimum feasible k
```

The exact implementation varies by problem, but the template is: find a lower/upper bound → test mid → keep the half that can still contain the optimal answer.

14 Reusable templates

Template 1 — Exact match

```
int low = 0, high = n - 1;
while (low <= high) {
    int mid = low + (high - low) / 2;
    if (a[mid] == target) return mid;
    if (a[mid] < target) low = mid + 1;
    else high = mid - 1;
}
return -1;
```

Template 2 — First valid position

```
int low = 0, high = n - 1, ans = n;
while (low <= high) {
    int mid = low + (high - low) / 2;
    if (condition(mid)) {
        ans = mid;
        high = mid - 1;
    } else {
        low = mid + 1;
    }
}
return ans;
```

Template 3 — Last valid position

```
int low = 0, high = n - 1, ans = -1;
while (low <= high) {
    int mid = low + (high - low) / 2;
    if (condition(mid)) {
        ans = mid;
        low = mid + 1;
    } else {
        high = mid - 1;
    }
}
return ans;
```

15 Common mistakes

- Using Binary Search on unsorted data: without an ordering/monotonic property, the left/right decision is not reliable.
- Wrong loop condition: for the closed interval $[low, high]$, use `while (low <= high)`.
- Wrong updates: use `mid + 1` and `mid - 1` when excluding `mid`.
- Overflow-prone midpoint: prefer `low + (high - low) / 2`.
- Returning too early with duplicates: first/last occurrence requires continuing after a match.
- Confusing index with value: decide whether the question asks for a position, a value, or a boundary.
- Ignoring empty input: `high` can become `-1`, which is fine if the loop is written correctly.

Debugging trick

Write a tiny table with `low`, `mid`, `high`, and `arr[mid]` for 3–5 iterations. Most pointer mistakes become obvious immediately.

16 Interview tip & problem-solving checklist

One interview tip

Before coding, say what makes the search space ordered. Then state your invariant: "If the target exists, it must remain inside [low, high]." This shows the interviewer you understand the algorithm rather than memorized syntax.

Checklist

- Is the array sorted?
- If not, is there another monotonic property?
- What exactly is the answer: index, first/last position, minimum, maximum?
- What are the initial low/high bounds?
- When condition is true, should I move left or right?
- What happens when $\text{low} > \text{high}$?
- Can mid calculation overflow?
- Have I tested empty, one-element, boundary, and absent-target cases?

17 Practice ladder

Solve these without looking at the solution first.

Level	Challenge	Skill
Beginner	Classic Binary Search	Exact search
Beginner	Search Insert Position	Lower boundary
Beginner+	First and Last Position	Duplicate boundaries
Intermediate	Sqrt(x)	Binary search on answer
Intermediate	Find Peak Element	Search using local property
Intermediate	Search in Rotated Sorted Array	Modified search
Advanced	Koko Eating Bananas	Answer-space search
Advanced	Capacity to Ship Packages Within D Days	Feasibility + boundary

Practice rule

For every problem: first identify the search space, then define the condition, then decide what happens when mid is too small/too large.

18 Quick revision sheet

Binary Search in 60 seconds

- Works on a sorted or monotonic search space.
- Maintain low, mid, high.
- Compare mid with target/condition.
- Discard half of the search space.
- Exact search → return on equality.
- First occurrence → match + move left.
- Last occurrence → match + move right.
- Answer search → binary search over candidate answers.
- Iterative time: $O(\log n)$, space: $O(1)$.
- Always test boundaries and empty input.

Core formula

$$\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$$

Mental model: "Don't search every possibility. Prove which half cannot contain the answer, remove it, and repeat."

19 What I'll learn tomorrow

Day 12 — Sorting Fundamentals

Tomorrow's progression can build naturally from searching into ordering:

- Why sorting matters in DSA.
- Stable vs unstable sorting.
- In-place vs extra-space sorting.
- Bubble Sort, Selection Sort and Insertion Sort.
- How sorting enables faster searching and two-pointer techniques.
- Time and space complexity comparison.

Series philosophy

Each day connects to the previous one. Binary Search becomes powerful because the data is ordered; the next step is to understand how we create that order efficiently.

20 Final takeaway

Binary Search is a mindset, not just a piece of code.

When a problem gives you an ordered search space—or lets you build a monotonic yes/no condition—ask whether you can eliminate half the possibilities at every step.

Remember this

Sorted / Monotonic → Define search space → Check mid → Eliminate half
→ Repeat → $O(\log n)$

If you can explain the invariant, trace the pointers, handle boundaries, and recognize the pattern in a new problem, you truly understand Binary Search.

DSA FROM SCRATCH TO ADVANCED • DAY 11

Binary Search — Implementation & Problems

Keep learning. Keep solving. Keep building. ☐